The Pure-Context-Window-Memory-as-Substrate Anti-Pattern: Standalone Formalization of a Tool-Agnosticism Failure Mode in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 6, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one specific failure mode — the architectural configuration in which the LLM's context window is itself positioned as the coordination substrate — as a standalone anti-pattern with independent operational content, distinct from the related failure modes at the partial-authority-migration position and at the separate-adjacent-substitute position.

Abstract

The CKS pattern's tool-agnosticism commitment requires that the substrate be instantiable in any environment meeting three minimal requirements — persistent structured state, human read/write access, and LLM access to substrate content (§7.1) — and not be tied to any specific tool, vendor, or technology stack. *Pure context-window memory as substrate* is the failure mode in which this commitment fails through a specific configuration: the LLM's own context window is architecturally positioned as the coordination substrate, with no separate persistent storage holding coordination state. Long-context LLM deployments holding all coordination state in the context window, in-context-learning architectures provisioning coordination state per invocation, prompt-as-substrate patterns encoding coordination structure entirely in prompt text, and serverless LLM-only deployments with no persistent state are common operational forms. This note states the four operational components of the anti-pattern, identifies the foundational architectural commitments it violates (most directly tool-agnosticism, with cascading violations of substrate-as-source-of-truth, the substrate-cell boundary, path retraceability, the determinism contract, AI-as-substrate-mediator, and human-governance), traces the failure mode, specifies the architectural correction, distinguishes the anti-pattern from four adjacent legitimate patterns, and provides an operational test for whether a given deployment exhibits it.

1. Why the anti-pattern needs to be formalized as standalone

CKS's tool-agnosticism commitment (§7.1) makes the substrate instantiable in any environment meeting three minimal requirements; it is the architectural promise that the pattern is not committed to any specific LLM, runtime, vendor, or context architecture. Pure context-window memory as substrate is the failure mode in which this commitment fails through a specific configuration: the LLM's own

context window plays the role the substrate is supposed to play, and the deployment's coordination architecture becomes the LLM's context architecture.

The failure mode is operationally common in 2024–2026 deployments. Long-context LLMs with token windows large enough to hold substantial coordination content make "fit everything in context" feasible; in-context-learning tooling encourages treating context as the deployment's knowledge; prompt-engineering disciplines position prompts as the operational primary; serverless AI architectures are commercially promoted; "AI-only" or "model-as-product" framings position the LLM as the entire system. Each of these is a positive technological development on its own terms. The anti-pattern emerges when they are applied to coordination state without preserving substrate as a persistent tool-independent architectural element separate from the LLM.

The standalone formalization is needed for three reasons. First, the configuration is the most direct violation of tool-agnosticism in the broader anti-pattern landscape: the deployment's coordination architecture *is* the LLM's context architecture, so migration across LLM vendors, model versions, context window sizes, or tokenization schemes requires re-architecting the entire deployment. Second, it is operationally distinct from neighboring failure modes — distinct from the case where substrate exists but the LLM's context becomes authoritative for some coordination questions, and distinct from the case where a separate adjacent component (a retrieval index, vector database, knowledge graph) is positioned as substrate substitute. Third, the framings under which the anti-pattern appears in practice — "we use a long-context LLM," "our architecture is AI-native," "we use in-context learning" — read as positive AI architecture without flagging the architectural consequence, so naming the configuration as a standalone anti-pattern gives downstream readers a precise specification of the failure mode and its correction.

2. The anti-pattern, defined precisely

A deployment exhibits *pure context-window memory as substrate* when its architecture has all four of the following operational components.

(a) The LLM's context window holds all coordination state, with no separate persistent substrate. The deployment has no architectural element separate from the LLM's context that persists coordination state across invocations. All coordination content — what is the case, what is current, what is in conflict, what rules apply, who has what authority — is loaded into the LLM's context for processing. Between invocations, this content exists only as material that can be reconstructed and reloaded into context.

(b) Coordination operations are organized around context-loading–LLM-processing cycles. The deployment's architectural primary is the cycle: construct context with relevant coordination content, invoke the LLM with the constructed context, act on the LLM's output. There is no persistent substrate that the cycle operates on; the cycle is the architecture, and what would otherwise be substrate state is reconstructed each time.

(c) The deployment's architecture is committed to the specific LLM's context window architecture. Design decisions, integration patterns, and operational characteristics are determined by the specific LLM's context window — its size, its formatting requirements, its attention mechanism, its

tokenization. The deployment's coordination capability is bound to those particulars.

(d) The deployment cannot operate without the specific LLM tool. Removed from the specific LLM, the deployment has no coordination architecture. There is no substrate to consult, no rules to apply outside the LLM's processing, and no state to preserve — all are encoded in the LLM-specific context architecture.

Components (a) and (b) name the architectural absence of persistent substrate; components (c) and (d) name the architectural commitment to the specific LLM tool. A deployment exhibiting all four exhibits the anti-pattern fully.

3. The architectural commitments violated

Pure context-window memory as substrate violates one foundational CKS commitment directly and cascades into extended violations of six others.

Tool-agnosticism (§7.1) — directly violated. The commitment that the substrate be instantiable in any environment meeting the three minimal requirements fails when the "substrate" is the LLM's specific context architecture. The deployment is not substrate-instantiable across environments; it is bound to one LLM's context model. The decomposition of tool-agnosticism into specific minimal requirements is violated across each requirement in turn.

Substrate as source of truth (§11.3, §6.2) — extended-violated. The commitment that the substrate be authoritative for coordination questions fails when no persistent substrate exists; there is nothing to be source of truth, and the categories of substrate-resident authoritative state cannot be carried by an ephemeral context window.

The substrate-cell boundary (§4.1) — extended-violated. The architectural separation between substrate (state) and cells (behavior) collapses when the LLM's own context — which is the cell's processing environment — is the substrate. Substrate's state-holding role is played by what should be cell-internal processing material.

Path retraceability (§3.1) — extended-violated. The retraceability machinery requires persistent substrate carrying provenance metadata; ephemeral context windows cannot host that machinery, and the provenance specification cannot operationally apply when the addressable artifact does not persist between invocations.

The determinism contract (§4.1, §6.2, §11.3) — extended-violated. Substrate operations must satisfy the contract's guarantees, including read determinism. When context-window construction is non-deterministic and the context window is the substrate, every coordination operation inherits that non-determinism; the same conceptual operation may produce different results across invocations because the context that "is" the substrate is differently constructed each time.

AI-as-substrate-mediator (§4.2) — extended-violated. The mediator role requires substrate as a separate architectural element from the LLM; the LLM's reads, writes, and processing operations happen *relative to* the substrate. When the LLM's context *is* the substrate, the mediator role collapses — there is no substrate separate from the mediator for the mediator to mediate over. The defining mediator properties — substrate as primary read source, no substrate-relevant state held outside

substrate — cannot apply when substrate-relevant state lives inside the LLM's own context.

Human-governance (§2.1, §3.3) — extended-violated. Human governance is exercised through rule authoring in substrate and direct override of substrate content; both require persistent substrate. When there is no persistent substrate, rules cannot be substrate-resident, override cannot land on substrate state, and the architectural mechanism for governance is not available. Composition requirements that depend on per-substrate human governance also extend-fail when there is no per-substrate object to govern.

The cascade is unusually broad: a single architectural configuration fails one commitment directly and extend-fails six across the foundational set. This breadth is what makes the configuration distinctively diagnostic — not a localized violation of one property but an architectural commitment that propagates across the pattern's foundational structure.

4. The failure mode

The cascade in §3 produces operationally specific consequences. Coordination state does not persist across invocations; what would be state must be reconstructed at each invocation, and because context construction is non-deterministic, "state" varies across invocations even when the conceptual situation has not. Humans cannot govern through the architectural mechanism the pattern names; without persistent substrate, rules cannot be substrate-resident, and the three rights — inspect, modify, override — are exercisable, if at all, against artifacts at a different architectural layer than the one the pattern identifies as governance-bearing. Retraceability machinery cannot exist; provenance metadata is substrate machinery, and an ephemeral context window cannot record it across invocations. Tool-agnosticism collapses entirely: vendor migrations, model-version upgrades, context-window-size changes, and tokenization changes require re-architecting, not because the implementation made tool-specific choices but because the substrate *is* the tool.

The cost-curve failure mode the source paper names as *context rot* (§6.2) is maximally instantiated. The §6.2 vocabulary describes capacity overflow, compaction loss, and goal drift under repeated compression as failure modes of the in-context alternative *to* substrate-bearing architectures; in pure context-window memory as substrate, the alternative *is* the architecture. There is no substrate for context to have drifted from; the entire coordination operates inside the regime the source paper identifies as the cost-curve failure region.

Recovery is operationally heavy. Unlike configurations in which substrate exists but partial authority has migrated, recovery here requires creating persistent substrate, migrating coordination state from context-construction patterns into substrate, re-architecting cells to read substrate as primary source and write substrate under rules, and re-positioning the LLM as a mediator within cells rather than as the architectural primary. The work is closer to building the architecture than to correcting a localized drift. The anti-pattern compounds with related failures: when context construction relies on LLM-produced summaries or LLM-derived structure, it compounds with the LLM-as-source-of-truth failure; when partial authority has previously migrated to LLM context with substrate still nominally present, progression toward pure context-window memory as substrate is the architectural endpoint.

5. The architectural correction

The correction operates through three foundational commitments together: tool-agnosticism (§7.1), substrate-as-source-of-truth (§11.3), and AI-as-substrate-mediator (§4.2). The substrate must exist as a separate architectural element with persistence across LLM invocations, model versions, vendor changes, and context architecture differences. Its representational form must be readable and writable by humans directly, addressable by cells across the substrate-cell boundary, and instantiable in any environment that meets the three minimal requirements; it must not be specific to any particular LLM's context architecture. Provenance metadata, governance affordances (the three rights at all times), and conflict-as-first-class-object handling are properties of this persistent substrate.

The LLM operates as a mediator within cells, not as the architectural primary. Cells read substrate as their primary source of state and write to substrate under human-authored orchestration rules; the LLM's context-window content is an input to cell reasoning, ephemeral and bounded by the cell's invocation. Long-context capabilities are useful as cell-reasoning capabilities — a cell may process more substrate content per invocation when the LLM has a larger context window — but the long context is not the substrate; it is a performance characteristic of cell processing. The substrate-cell boundary is preserved: substrate state is persistent and tool-independent; cells are processing units that may include LLM operations within rule-governed contexts. The deployment is portable across LLM vendors, model versions, and context window sizes because substrate is independent of all three.

The correction reframes two specific architectural framings that often produce the anti-pattern. *Long-context-LLM* deployments are legitimate when the long context is positioned as a cell-reasoning capability with substrate remaining the architectural primary; the same deployments instantiate the anti-pattern when the long context is positioned as the substrate. *AI-only* or *model-as-product* deployments are legitimate when the AI product operates within the cell-rule architecture with substrate as architectural primary; they instantiate the anti-pattern when the AI product is positioned as replacing substrate. The architectural test is whether substrate exists as a separate persistent tool-independent element, not whether the deployment uses long-context LLMs or markets itself as AI-native.

6. What the anti-pattern is NOT

The anti-pattern is precise about which configurations it names. Stating which adjacent configurations are not the anti-pattern is what keeps the standalone framing from drifting into overcorrection.

Not LLM context as input to cell reasoning, with substrate as primary. A cell that loads substrate-derived content into the LLM's context for processing is exercising the AI-as-substrate-mediator role correctly when substrate exists as the persistent tool-independent primary and the cell reads substrate before constructing context. The anti-pattern is the architectural inversion in which context *is* the substrate.

Not long-context cell processing for performance. Cells that take advantage of larger LLM context windows to process more substrate content per invocation — reducing the number of substrate-consultation cycles — are operating within the architecture; the long context is a cell-performance optimization, and substrate remains the architectural primary. The anti-pattern is long-context-as-substrate, not long-context-for-cell-performance.

Not in-context learning for cell-specific behavior under rules. Cells that use in-context learning patterns — providing examples, demonstrations, or constraints in context to guide LLM behavior — are operating within the cell-rule architecture when substrate remains the architectural primary and the in-context material is bounded to the cell's invocation. The anti-pattern is in-context-learning-as-substrate, not in-context learning within cells.

Not prompt-engineered cell behavior with substrate as authoritative reference. Deployments that use prompt engineering to shape cell-specific LLM behavior under human-authored rules are operating within the architecture when substrate remains authoritative for coordination questions. The anti-pattern is prompt-as-substrate, not prompt-engineering-within-cells.

The four configurations describe the legitimate ways an LLM's context window participates in a CKS deployment. The anti-pattern is the specific commitment that elevates the context window from a cell-processing input to the architectural substrate itself.

7. Operational test

A deployment exhibits pure context-window memory as substrate if all four of the following hold during its existence.

1. The LLM's context window holds all coordination state, with no separate persistent substrate carrying that state across invocations.

2. Coordination operations are organized around context-loading–LLM-processing cycles as the architectural primary, rather than around persistent substrate that cells read and write.

3. The deployment's design decisions — schema, integration patterns, operational characteristics — are determined by the specific LLM's context window architecture, such that they would change materially under a different LLM.

4. The deployment cannot operate without the specific LLM tool: removed from it, no coordination architecture remains.

Three sharpening properties refine the test for review purposes. *Persistent-substrate-existence* — verify operationally that substrate exists as a persistent architectural element separate from the LLM, with provenance metadata and governance affordances available at the substrate level; absence of such an element indicates the anti-pattern. *Deployment-LLM-portability* — verify operationally that the deployment can operate with a different LLM tool without architectural changes; inability to migrate without re-architecting indicates the anti-pattern. *Governance-mechanism-existence* — verify operationally that humans can govern through rule authoring in substrate (the architectural mechanism the pattern commits to) rather than through prompt construction or LLM-vendor-specific affordances; governance only through the latter indicates the anti-pattern.

A one-sentence form for quick classification: *if a deployment positions the LLM's context window as the architectural substrate — with no separate persistent storage for coordination state, with coordination operating through cycles of loading content into LLM context and the LLM processing the context, and with the deployment specifically committed to the LLM's context architecture such that migration across LLM vendors, model versions, or context window sizes requires re-architecting — the*

deployment exhibits pure context-window memory as substrate, and tool-agnosticism is directly violated, with the cascade through substrate-as-source-of-truth, the substrate-cell boundary, path retraceability, the determinism contract, AI-as-substrate-mediator, and human-governance described in §3.

8. Why naming the anti-pattern as standalone matters

Deployments under pressure to deliver AI products with sophisticated coordination capabilities default toward pure context-window memory as substrate steadily and for legible reasons. Long-context LLMs make "fit the coordination state in context" feasible at scales it was not previously; AI-native and model-as-product framings position the LLM as the deployment's architectural primary; in-context learning and prompt-engineering disciplines treat context content as the deployment's knowledge; serverless and stateless commercial architectures economize on persistent infrastructure. The drift is steady because the framings under which it occurs read as positive AI architecture; the architectural consequence — that substrate has been committed to LLM-specific tool architecture and the foundational tool-agnosticism commitment fails — is not visible from the framing alone.

Naming the configuration as a standalone anti-pattern, with the four operational components in §2, the cascade of violations in §3, the failure mode in §4, the architectural correction in §5, the four adjacent-pattern distinctions in §6, and the operational test in §7, gives downstream readers a precise specification of the failure and its correction. It separates the anti-pattern from related failure modes at neighboring architectural positions and preserves the architectural distinction between long-context-LLM-as-cell-capability (legitimate, under substrate-as-primary) and long-context-LLM-as-substrate (the anti-pattern), which is the operational test that matters in 2024–2026 deployments where long-context capabilities are widespread and the framing under which they are used does not reliably distinguish the two.

Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should treat pure context-window memory as substrate in the sense formalized here. Work that deploys long-context LLMs, in-context learning, prompt engineering, or AI-native architectures inside CKS deployments without preserving substrate as a persistent tool-independent architectural element is using the LLM's context window as the substrate, and the architectural consequences described in §3 and §4 follow.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Pure-Context-Window-Memory-as-Substrate Anti-Pattern: Standalone Formalization of a Tool-Agnosticism Failure Mode in the Coordination Knowledge Substrate Pattern.* May 6, 2026. ORCID: 0009-0004-8065-3235.